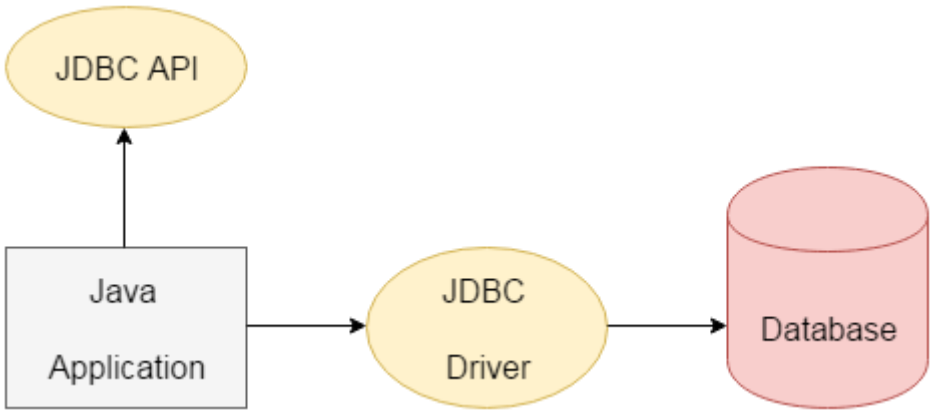
# JDBC Notes

JDBC is a Java API that is used to connect and execute the query to the database. JDBC API uses JDBC drivers to connect to the database. JDBC API can be used to access tabular data stored into any relational database.




## JDBC Driver

JDBC Driver is a software component that enables java application to interact with the database. There are 4 types of JDBC drivers:

1. JDBC-ODBC bridge driver
2. Native-API driver (partially java driver)
3. Network Protocol driver (fully java driver)
4. Thin driver (fully java driver)

### 1. JDBC-ODBC bridge driver

The JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. This is now discouraged because of thin driver.

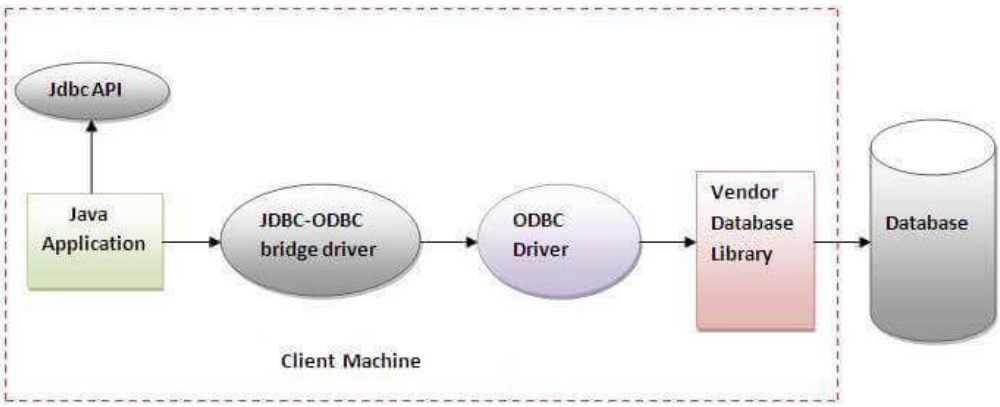



Figure- JDBC-ODBC Bridge Driver

Oracle does not support the JDBC-ODBC Bridge from Java 8. Oracle recommends that you use JDBC drivers provided by the vendor of your database instead of the JDBC-ODBC Bridge.

**Advantages:**

- Easy to use.
- Can be easily connected to any database.

**Disadvantages:**

- Performance degraded because JDBC method call is converted into the ODBC function calls.
- The ODBC driver needs to be installed on the client machine.

**2) Native-API driver**

The Native API driver uses the client-side libraries of the database. The driver converts JDBC method calls into native calls of the database API. It is not written entirely in java.

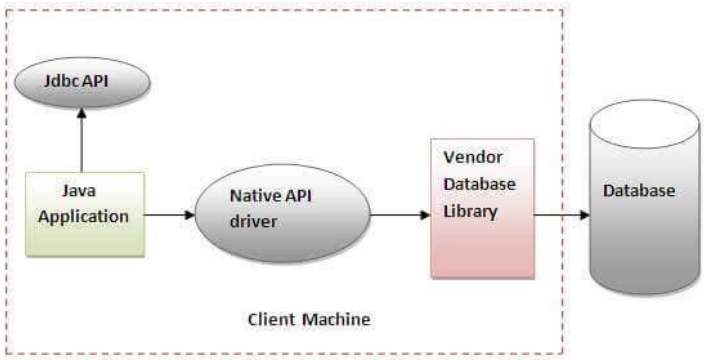



Figure- Native API Driver

**Advantage:**

- Performance upgraded than JDBC-ODBC bridge driver.

**Disadvantage:**

- The Native driver needs to be installed on the each client machine.
- The Vendor client library needs to be installed on client machine.

**3) Network Protocol driver**

The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is fully written in java.

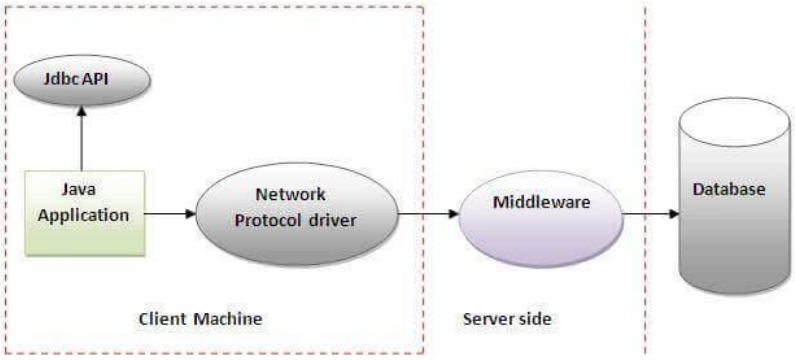



Figure- Network Protocol Driver

**Advantage:**

- No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.

**Disadvantages:**

- Network support is required on client machine.
- Requires database-specific coding to be done in the middle tier.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

**4) Thin driver**

The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as thin driver. It is fully written in Java language.

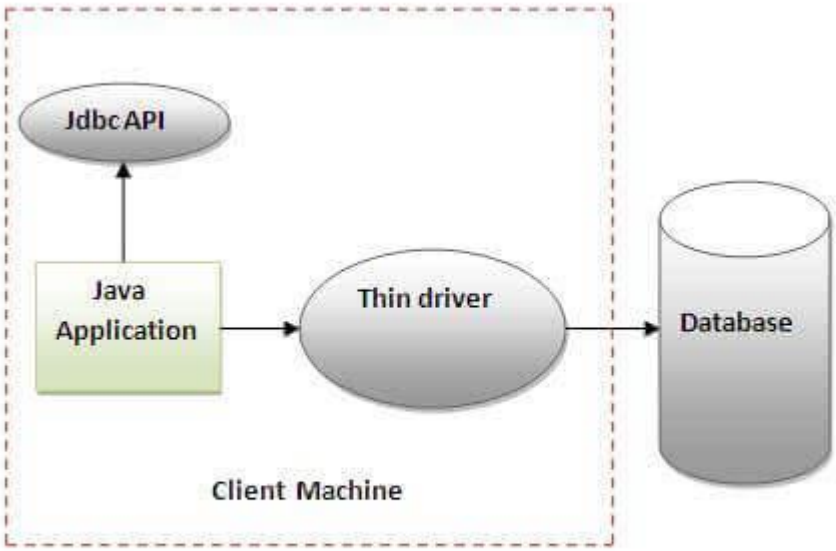



Figure- Thin Driver

**Advantage:**

- Better performance than all other drivers.
- No software is required at client side or server side.

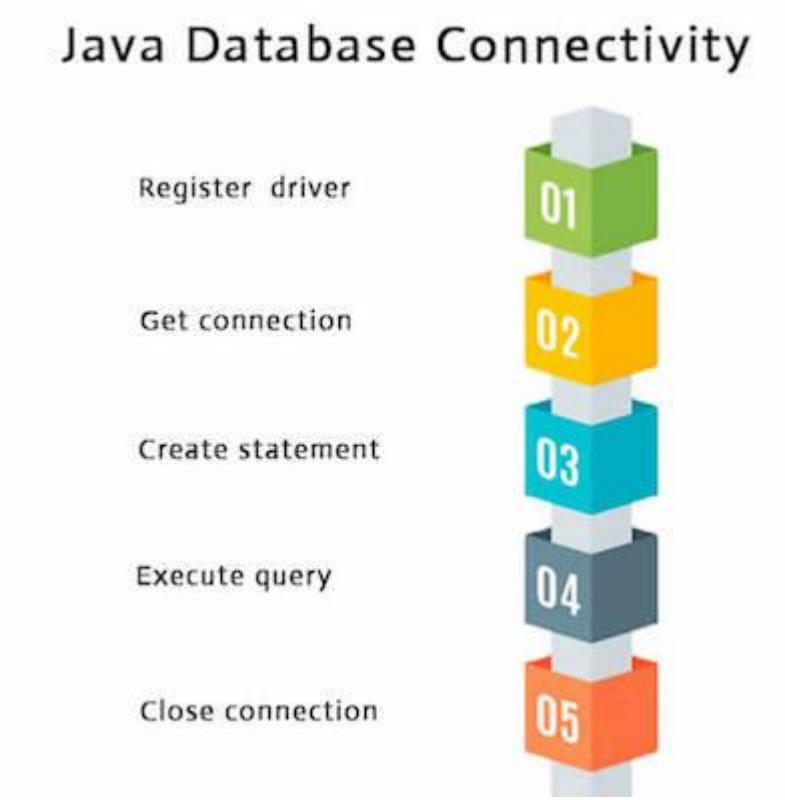
**Disadvantage:**

- Drivers depend on the Database.

# Java Database Connectivity with 6 Steps

There are 6 steps to connect any java application with the database using JDBC. These steps are as follows:

- Importing JDBC package
- Load/Register the Driver class
- Establish the connection
- Create statement
- Execute queries
- Close connection




**1. Importing JDBC Package.**

We need to import java.sql.* package of JDBC to use all the classes and Interfaces available in that package to establish the communication with any database vendor through java application.

Ex: import java.sql.*;

**2. Loading/Registering the driver class:**

**Loading:**

The forName() method of the Class class is used to Load the driver class. This method is used to load the driver class dynamically.

Consider the following example to load OracleDriver class.

Ex: Class.forName("oracle.jdbc.driver.OracleDriver");

**Registering:**

The registerDriver() method of DriverManager class is used to register the dirver class object.

Consider the following example to register OracleDriver class.

Ex: oracle.jdbc.driver.OracleDriver od= new oracle.jdbc.driver.OracleDriver();

DriverManager.registerDriver(od);

**3. Establishing the connection:**

The getConnection() method of DriverManager class is used to establish the connection with the database. The syntax of the getConnection() method is given below.

1) **public static** Connection getConnection(String url)**throws** SQLException
2) **public static** Connection getConnection(String url,String name,String password)
**throws** SQLException

Consider the following example to establish the connection with the Oracle database.

```
Connection con=
        DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","password");
```

**4. Creating the statement:**

The createStatement() method of Connection interface is used to create the Statement. The object of the Statement is responsible for executing queries with the database.

**public** Statement createStatement()**throws** SQLException

consider the following example to create the statement object

Ex: Statement stmt=con.createStatement();

**5. Executing the queries:**

The executeQuery() method of Statement interface is used to execute queries to the database. This method returns the object of ResultSet that can be used to get all the records of a table.

Syntax of executeQuery() method is given below.

**public** ResultSet executeQuery(String sql)**throws** SQLException

Example to execute the query

```
ResultSet rs=stmt.executeQuery("select * from emp");
while(rs.next()){
System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

However, to perform the insert and update operations in the database, executeUpdate() method is used which returns the boolean value to indicate the successful completion of the operation.

**6. Closing connection:**

By closing connection, object statement and ResultSet will be closed automatically. The close() method of Connection interface is used to close the connection.

Syntax of close() method is given below.

**public void** close()**throws** SQLException

Consider the following example to close the connection.

con.close();

## JDBC API components

The java.sql package contains following interfaces and classes for JDBC API.

**Interfaces:**

- **Connection:** The Connection object is created by using getConnection() method of DriverManager class. DriverManager is the factory for connection.

- **Statement:** The Statement object is created by using createStatement() method of Connection class. The Connection interface is the factory for Statement.

- **PreparedStatement:** The PrepareStatement object is created by using prepareStatement() method of Connection class. It is used to execute the parameterized query.

- **ResultSet:** The object of ResultSet maintains a cursor pointing to a row of a table. Initially, cursor points before the first row. The executeQuery() method of Statement interface returns the ResultSet object.

- **ResultSetMetaData:** The object of ResultSetMetaData interface cotains the information about the data (table) such as numer of columns, column name, column type, etc. The getMetaData() method of ResultSet returns the object of ResultSetMetaData.

- **DatabaseMetaData:** DatabaseMetaData interface provides methods to get metadata of a database such as the database product name, database product version, driver name, name of the total number of tables, the name of the total number of views, etc. The getMetaData() method of Connection interface returns the object of DatabaseMetaData.

- **CallableStatement:** CallableStatement interface is used to call the stored procedures and functions. We can have business logic on the database through the use of stored procedures and functions that will make the performance better because these are precompiled. The prepareCall() method of Connection interface returns the instance of CallableStatement.

**Classes:**

- **DriverManager:** The DriverManager class acts as an interface between the user and drivers. It keeps track of the drivers that are available and handles establishing a connection between a database and the appropriate driver. It contains several methods to keep the interaction between the user and drivers.

- **Blob:** Blob stands for the binary large object. It represents a collection of binary data stored as a single entity in the database management system.

- **Clob:** Clob stands for Character large object. It is a data type that is used by various database management systems to store character files. It is similar to Blob except for the difference that BLOB represent binary data such as images, audio and video files, etc. whereas Clob represents character stream data such as character files, etc.

- **SQLException** It is an Exception class which provides information on database access errors.

## Java Database Connectivity with Oracle

To connect java application with the oracle database, we need to follow 5 following steps. In this example, we are using Oracle 10g as the database. So we need to know following information for the oracle database:

1. **Driver class:** The driver class for the oracle database is **oracle.jdbc.driver.OracleDriver**.
2. **Connection URL:** The connection URL for the oracle10G database is **jdbc:oracle:thin:@localhost:1521:xe** where jdbc is the API, oracle is the database, thin is the driver, localhost is the server name on which oracle is running, we may also use IP address, 1521 is the port number and XE is the Oracle service name. You may get all these information from the tnsnames.ora file.
3. **Username:** The default username for the oracle database is **system**.
4. **Password:** It is the password given by the user at the time of installing the oracle database.

**Create a Table**

Before establishing connection, let's first create a table in oracle database. Following is the SQL query to create a table.

```
create table emp(id number(10),name varchar2(40),age number(3));
```

**Example to Connect Java Application with Oracle database**

In this example, we are connecting to an Oracle database and getting data from **emp** table.
Here, **system** and **oracle** are the username and password of the Oracle database.

```
import java.sql.*;
class OracleCon
{
	public static void main(String args[])
	{
		Try
		{
			//step1 load the driver class
			Class.forName("oracle.jdbc.driver.OracleDriver");

			//step2 create  the connection object
			Connection con=DriverManager.getConnection(
					"jdbc:oracle:thin:@localhost:1521:xe","system","oracle
					");

			//step3 create the statement object
			Statement stmt=con.createStatement();

			//step4 execute query
			ResultSet rs=stmt.executeQuery("select * from emp");
			while(rs.next())
			System.out.println(rs.getInt(1)+"  "+rs.getString(2)+"  "+rs.getString(3));

			//step5 close the connection object
			con.close();

		}catch(Exception e)
		{
			System.out.println(e);
		}
	}
}
```

The above example will fetch all the records of emp table.

To connect java application with the Oracle database ojdbc14.jar file is required to be loaded.

**Two ways to load the jar file:**

1. paste the ojdbc14.jar file in jre/lib/ext folder
2. set classpath

**1) Paste the ojdbc14.jar file in JRE/lib/ext folder:**

Firstly, search the ojdbc14.jar file then go to JRE/lib/ext folder and paste the jar file here.

**2) set classpath:**

There are two ways to set the classpath:

- temporary

- permanent

**How to set the temporary classpath:**

Firstly, search the ojdbc14.jar file then open command prompt and write:

C:>set classpath=c:\folder\ojdbc14.jar;.;

**How to set the permanent classpath:**

Go to environment variable then click on new tab. In variable name write **classpath** and in variable value paste the path to ojdbc14.jar by appending ojdbc14.jar;.;

C:\oraclexe\app\oracle\product\10.2.0\server\jdbc\lib\ojdbc14.jar;.;

# JDBC Package

## DriverManager class

The DriverManager class acts as an interface between user and drivers. It keeps track of the drivers that are available and handles establishing a connection between a database and the appropriate driver. The DriverManager class maintains a list of Driver classes that have registered themselves by calling the method DriverManager.registerDriver().

**Useful methods of DriverManager class**

| Method | Description |
|---|---|
| **1) public static void registerDriver(Driver driver):** | is used to register the given driver with DriverManager. |
| **2) public static void deregisterDriver(Driver driver):** | is used to deregister the given driver (drop the driver from the list) with DriverManager. |
| **3) public static Connection getConnection(String url):** | is used to establish the connection with the specified url. |
| **4) public static Connection getConnection(String url,String userName,String password):** | is used to establish the connection with the specified url, username and password. |

## Connection interface

A Connection is the session between java application and database. The Connection interface is a factory of Statement, PreparedStatement, and DatabaseMetaData i.e. object of Connection can be used to get the object of Statement and DatabaseMetaData. The Connection interface provide many methods for transaction management like commit(), rollback() etc.

**Commonly used methods of Connection interface:**

**1) public Statement createStatement():** creates a statement object that can be used to execute SQL queries.

**2) public Statement createStatement(int resultSetType,int resultSetConcurrency):** Creates a Statement object that will generate ResultSet objects with the given type and concurrency.

**3) public void setAutoCommit(boolean status):** is used to set the commit status.By default it is true.

**4) public void commit():** saves the changes made since the previous commit/rollback permanent.

**5) public void rollback():** Drops all changes made since the previous commit/rollback.

**6) public void close():** closes the connection and Releases a JDBC resources immediately.

## Statement interface

The **Statement interface** provides methods to execute queries with the database. The statement interface is a factory of ResultSet i.e. it provides factory method to get the object of ResultSet.

**Commonly used methods of Statement interface:**

The important methods of Statement interface are as follows:

**1) public ResultSet executeQuery(String sql):** is used to execute SELECT query. It returns the object of ResultSet.

**2) public int executeUpdate(String sql):** is used to execute specified query, it may be create, drop, insert, update, delete etc.

**3) public boolean execute(String sql):** is used to execute queries that may return multiple results.

**4) public int[] executeBatch():** is used to execute batch of commands.

**Example of Statement interface**

Let's see the simple example of Statement interface to insert, update and delete the record.

```
import java.sql.*;
class FetchRecord
{
	public static void main(String args[])throws Exception
	{
		Class.forName("oracle.jdbc.driver.OracleDriver");
		Connection con=
		DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
		Statement stmt=con.createStatement();

		//stmt.executeUpdate("insert into emp765 values(33,'Irfan',50000)");
		//int result=stmt.executeUpdate("update emp765 set name='Vimal',salary=10000 where id=3
		3");
		int result=stmt.executeUpdate("delete from emp765 where id=33");
		System.out.println(result+" records affected");
		con.close();
```

```
        }
}
```

## PreparedStatement interface

The PreparedStatement interface is a subinterface of Statement. It is used to execute parameterized query.

Let's see the example of parameterized query:

```
String sql="insert into emp values(?,?,?)";
```

As you can see, we are passing parameter (?) for the values. Its value will be set by calling the setter methods of PreparedStatement.

**Why use PreparedStatement?**

**Improves performance**: The performance of the application will be faster if you use PreparedStatement interface because query is compiled only once.

**How to get the instance of PreparedStatement?**

The prepareStatement() method of Connection interface is used to return the object of PreparedStatement. Syntax:

```
public PreparedStatement prepareStatement(String query)throws SQLException{}
```

**Methods of PreparedStatement interface**

The important methods of PreparedStatement interface are given below:

| Method | Description |
|---|---|
| **public void setInt(int paramIndex, int value)** | sets the integer value to the given parameter index. |
| **public void setString(int paramIndex, String value)** | sets the String value to the given parameter index. |
| **public void setFloat(int paramIndex, float value)** | sets the float value to the given parameter index. |
| **public void setDouble(int paramIndex, double value)** | sets the double value to the given parameter index. |
| **public int executeUpdate()** | executes the query. It is used for create, drop, insert, update, delete etc. |
| **public ResultSet executeQuery()** | executes the select query. It returns an instance of ResultSet. |

**Example of PreparedStatement interface that inserts the record**

First of all create table as given below:

create table emp(id number(10),name varchar2(50));

Now insert records in this table by the code given below:

```
import java.sql.*;
class InsertPrepared
{
	public static void main(String args[])
	{
		Try
		{
			Class.forName("oracle.jdbc.driver.OracleDriver");

			Connection  con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe
			","system","oracle");

			PreparedStatement stmt=con.prepareStatement("insert into Emp values(?,?)");
			stmt.setInt(1,101);//1 specifies the first parameter in the query
			stmt.setString(2,"Ratan");

			int i=stmt.executeUpdate();
			System.out.println(i+" records inserted");

			con.close();

		}
		catch(Exception e)
		{
			 System.out.println(e);
		}
	}
}
```

## CallableStatement Interface

CallableStatement interface is used to call the **stored procedures and functions**.

We can have business logic on the database by the use of stored procedures and functions that will make the performance better because these are precompiled.

Suppose you need the get the age of the employee based on the date of birth, you may create a function that receives date as the input and returns age of the employee as the output.

**How to get the instance of CallableStatement?**

The prepareCall() method of Connection interface returns the instance of CallableStatement. Syntax is given below:

**public** CallableStatement prepareCall("{ call procedurename(?,?...?)}");

The example to get the instance of CallableStatement is given below:
CallableStatement stmt=con.prepareCall("{call myprocedure(?,?)}");

It calls the procedure myprocedure that receives 2 arguments.

**Full example to call the stored procedure using JDBC**
To call the stored procedure, you need to create it in the database. Here, we are assuming that stored procedure looks like this.

```
create or replace procedure "INSERTR"
(id IN NUMBER,
name IN VARCHAR2)
is
begin
insert into user420 values(id,name);
end;
/
```

The table structure is given below:

```
create table user420(id number(10), name varchar2(200));
```

In this example, we are going to call the stored procedure INSERTR that receives id and name as the parameter and inserts it into the table user420. Note that you need to create the user420 table as well to run this application.

```
import java.sql.*;
public class Proc
{
	public static void main(String[] args) throws Exception
	{

		Class.forName("oracle.jdbc.driver.OracleDriver");
		Connection con=DriverManager.getConnection(
					"jdbc:oracle:thin:@localhost:1521:xe","system","oracle");

		CallableStatement stmt=con.prepareCall("{call insertR(?,?)}");
		stmt.setInt(1,1011);
		stmt.setString(2,"Amit");
		stmt.execute();

		System.out.println("success");
	}
}
```

Now check the table in the database, value is inserted in the user420 table.

**Example to call the function using JDBC**

In this example, we are calling the sum4 function that receives two input and returns the sum of the given number. Here, we have used the **registerOutParameter** method of CallableStatement interface, that registers the output parameter with its corresponding type. It provides information to the CallableStatement about the type of result being displayed.

The **Types** class defines many constants such as INTEGER, VARCHAR, FLOAT, DOUBLE, BLOB, CLOB etc.

Let's create the simple function in the database first.

```
create or replace function sum4
(n1 in number,n2 in number)
return number
is
temp number(8);
begin
temp :=n1+n2;
return temp;
end;
/
```

Now, let's write the simple program to call the function.

```
import java.sql.*;

public class FuncSum
{
	public static void main(String[] args) throws Exception
	{

		Class.forName("oracle.jdbc.driver.OracleDriver");
		Connection con=DriverManager.getConnection(
					"jdbc:oracle:thin:@localhost:1521:xe","system","oracle");

		CallableStatement stmt=con.prepareCall("{?= call sum4(?,?)}");
		stmt.setInt(2,10);
		stmt.setInt(3,43);
		stmt.registerOutParameter(1,Types.INTEGER);
		stmt.execute();

		System.out.println(stmt.getInt(1));

	}
}
```

## ResultSet interface

The object of ResultSet maintains a cursor pointing to a row of a table. Initially, cursor points to before the first row.

**Note:** By default, ResultSet object can be moved forward only and it is not updatable.

But we can make this object to move forward and backward direction by passing either TYPE_SCROLL_INSENSITIVE or TYPE_SCROLL_SENSITIVE in createStatement(int,int) method as well as we can make this object as updatable by:

Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE);

**Commonly used methods of ResultSet interface**

| | |
|---|---|
| **1) public boolean next():** | is used to move the cursor to the one row next from the current position. |
| **2) public boolean previous():** | is used to move the cursor to the one row previous from the current position. |
| **3) public boolean first():** | is used to move the cursor to the first row in result set object. |
| **4) public boolean last():** | is used to move the cursor to the last row in result set object. |
| **5) public boolean absolute(int row):** | is used to move the cursor to the specified row number in the ResultSet object. |
| **6) public boolean relative(int row):** | is used to move the cursor to the relative row number in the ResultSet object, it may be positive or negative. |
| **7) public int getInt(int columnIndex):** | is used to return the data of specified column index of the current row as int. |
| **8) public int getInt(String columnName):** | is used to return the data of specified column name of the current row as int. |
| **9) public String getString(int columnIndex):** | is used to return the data of specified column index of the current row as String. |
| **10) public String getString(String columnName):** | is used to return the data of specified column name of the current row as String. |

**Example of Scrollable ResultSet**

Let's see the simple example of ResultSet interface to retrieve the data of 3rd row.

```
import java.sql.*;
class FetchRecord
{
    public static void main(String args[])throws Exception
    {
```

```
            Class.forName("oracle.jdbc.driver.OracleDriver");
            Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","syste
            m","oracle");
            Statement stmt=con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
            ResultSet.CONCUR_UPDATABLE);
            ResultSet rs=stmt.executeQuery("select * from emp765");

            //getting the record of 3rd row
            rs.absolute(3);
            System.out.println(rs.getString(1)+" "+rs.getString(2)+" "+rs.getString(3));

            con.close();
        }
}
```

## ResultSetMetaData Interface

The metadata means data about data i.e. we can get further information from the data.

If you have to get metadata of a table like total number of column, column name, column type etc. , ResultSetMetaData interface is useful because it provides methods to get metadata from the ResultSet object.

**Commonly used methods of ResultSetMetaData interface**

| Method | Description |
|---|---|
| **public int getColumnCount()throws SQLException** | it returns the total number of columns in the ResultSet object. |
| **public String getColumnName(int index)throws SQLException** | it returns the column name of the specified column index. |
| **public String getColumnTypeName(int index)throws SQLException** | it returns the column type name for the specified index. |
| **public String getTableName(int index)throws SQLException** | it returns the table name for the specified column index. |

**How to get the object of ResultSetMetaData:**

The getMetaData() method of ResultSet interface returns the object of ResultSetMetaData. Syntax:

```
public ResultSetMetaData getMetaData()throws SQLException
```

**Example of ResultSetMetaData interface :**

```
import java.sql.*;
class Rsmd
{
        public static void main(String args[])
        {
                try{
```

```
                Class.forName("oracle.jdbc.driver.OracleDriver");
                Connection con=DriverManager.getConnection(
                                "jdbc:oracle:thin:@localhost:1521:xe","system","oracle");

                PreparedStatement ps=con.prepareStatement("select * from emp");
                ResultSet rs=ps.executeQuery();
                ResultSetMetaData rsmd=rs.getMetaData();

                System.out.println("Total columns: "+rsmd.getColumnCount());
                System.out.println("Column Name of 1st column: "+rsmd.getColumnName(1));
                System.out.println("Column Type Name of 1st column: "
                                                        +rsmd.getColumnTypeName(1));

                con.close();
                }catch(Exception e)
                {
                        System.out.println(e);
                }
        }
}
```

## Batch Processing in JDBC

Instead of executing a single query, we can execute a batch (group) of queries. It makes the performance fast.

The java.sql.Statement and java.sql.PreparedStatement interfaces provide methods for batch processing.

***Advantage of Batch Processing***

Fast Performance

***Methods of Statement interface***

The required methods for batch processing are given below:

| Method | Description |
|---|---|
| **void addBatch(String query)** | It adds query into batch. |
| **int[] executeBatch()** | It executes the batch of queries. |

**Example of batch processing in jdbc**

```
import java.sql.*;
class FetchRecords
{
        public static void main(String args[])throws Exception
        {
                Class.forName("oracle.jdbc.driver.OracleDriver");
```

```
            Connection con=
            DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
            con.setAutoCommit(false);

            Statement stmt=con.createStatement();
            stmt.addBatch("insert into user420 values(190,'abhi',40000)");
            stmt.addBatch("insert into user420 values(191,'umesh',50000)");

            stmt.executeBatch();//executing the batch

            con.commit();
            con.close();
        }
}
```

If you see the table user420, two records has been added.

## Important Interview Questions

**What are the JDBC statements?**

In JDBC, Statements are used to send SQL commands to the database and receive data from the database. There are various methods provided by JDBC statements such as execute(), executeUpdate(), executeQuery, etc. which helps you to interact with the database.

There is three type of JDBC statements given in the following table.

| Statements | Explanation |
|---|---|
| **Statement** | Statement is the factory for resultset. It is used for general purpose access to the database. It executes a static SQL query at runtime. |
| **PreparedStatement** | The PreparedStatement is used when we need to provide input parameters to the query at runtime. |
| **CallableStatement** | CallableStatement is used when we need to access the database stored procedures. It can also accept runtime parameters. |

**What are the differences between Statement and PreparedStatement interface?**

| Statement | PreparedStatement |
|---|---|
| The Statement interface provides methods to execute queries with the database. The statement interface is a factory of ResultSet; i.e., it provides the factory method to get the object of ResultSet. | The PreparedStatement interface is a subinterface of Statement. It is used to execute the parameterized query. |
| In the case of Statement, the query is compiled each time we run the program. | In the case of PreparedStatement, the query is compiled only once. |

| | |
|---|---|
| The Statement is mainly used in the case when we need to run the static query at runtime. | PreparedStatement is used when we need to provide input parameters to the query at runtime. |

**What are the benefits of PreparedStatement over Statement?**

The benefits of using PreparedStatement over Statement interface is given below.

- The PreparedStatement performs faster as compare to Statement because the Statement needs to be compiled everytime we run the code whereas the PreparedStatement compiled once and then execute only on runtime.
- PreparedStatement can execute Parameterized query whereas Statement can only run static queries.
- The query used in PreparedStatement is appeared to be similar every time. Therefore, the database can reuse the previous access plan whereas, Statement inline the parameters into the String, therefore, the query doesn't appear to be same everytime which prevents cache reusage.
- PreparedStatement can prevent SQL Injection attacks.

**What are the differences between execute, executeQuery, and executeUpdate?**

| **execute** | **executeQuery** | **executeUpdate** |
|---|---|---|
| The execute method can be used for any SQL statements(Select and Update both). | The executeQuery method can be used only with the select statement. | The executeUpdate method can be used to update/delete/insert operations in the database. |
| The execute method returns a boolean type value where true indicates that the ResultSet s returned which can later be extracted and false indicates that the integer or void value is returned. | The executeQuery() method returns a ResultSet object which contains the data retrieved by the select statement. | The executeUpdate() method returns an integer value representing the number of records affected where 0 indicates that query returns nothing. |

**What are the different types of ResultSet?**

ResultSet is categorized by the direction of the reading head and sensitivity or insensitivity of the result provided by it. There are three general types of ResultSet.

| Type | Description |
|---|---|
| ResultSet.TYPE_Forward_ONLY | The cursor can move in the forward direction only. |
| ResultSet.TYPE_SCROLL_INSENSITIVE | The cursor can move in both the direction (forward and backward). The ResultSet is not sensitive to the changes made by the others to the database. |
| ResultSet.TYPE_SCROLL_SENSITIVE | The cursor can move in both the direction. The ResultSet is sensitive to the changes made by the others to the database. |

**Concurrency of ResultSet**

The possible RSConcurrency are given below. If you do not specify any Concurrency type, you will automatically get one that is CONCUR_READ_ONLY.

| Concurrency | Description |
|---|---|
| **ResultSet.CONCUR_READ_ONLY** | Creates a read-only result set. This is the default |
| **ResultSet.CONCUR_UPDATABLE** | Creates an updateable result set. |